

SESSION 6 - ORDER BY + Sorting

TASKS

Task 1: Write an SQL query to display all products from a 'products' table and sort them by price in ascending order, similar to how Flipkart lists items from lowest to highest price.

Query / Answer:

```
CREATE TABLE products (  
  product_id INT AUTO_INCREMENT PRIMARY KEY,  
  product_name VARCHAR(100),  
  category VARCHAR(50),  
  brand VARCHAR(50),  
  price DECIMAL(10,2),  
  stock INT  
);
```

```
INSERT INTO products (product_name, category, brand, price, stock) VALUES  
(  
'iPhone 15', 'Mobile', 'Apple', 79999.00, 20),  
(  
'Galaxy S24', 'Mobile', 'Samsung', 69999.00, 15),  
(  
'Redmi Note 13', 'Mobile', 'Xiaomi', 18999.00, 50),  
(  
'OnePlus 12', 'Mobile', 'OnePlus', 54999.00, 25),  
(  
'Realme Narzo 70', 'Mobile', 'Realme', 15999.00, 40),  
(  
'Boat Rockerz 450', 'Headphones', 'Boat', 1499.00, 100),  
(  
'Sony WH-1000XM5', 'Headphones', 'Sony', 29999.00, 10),  
(  
'Dell Inspiron 15', 'Laptop', 'Dell', 58999.00, 12);
```

```
select * from products order by price;
```

SESSION 6 - ORDER BY + Sorting

Image / Screenshot:

The screenshot displays a database management interface with a menu bar (File, Edit, View, Query, Database, Server, Tools, Scripting, Help) and a toolbar. The left sidebar shows a tree view of database objects under the 'CHEMAS' schema, including 'classwork', 'music_streaming_app', 'product', and 'sys'. The main area shows a SQL query in a text editor:

```
18 ('Realme Narzo 70', 'Mobile', 'Realme', 15999.00, 40),  
19 ('Boat Rockerz 450', 'Headphones', 'Boat', 1499.00, 100),  
20 ('Sony WH-1000XM5', 'Headphones', 'Sony', 29999.00, 10),  
21 ('Dell Inspiron 15', 'Laptop', 'Dell', 58999.00, 12);  
22  
23  
24 • select * from products order by price;
```

Below the query editor, the 'Result Grid' shows the output of the query, sorted by price. The table has columns: product_id, product_name, category, brand, price, and stock. The data is as follows:

product_id	product_name	category	brand	price	stock
6	Boat Rockerz 450	Headphones	Boat	1499.00	100
5	Realme Narzo 70	Mobile	Realme	15999.00	40
3	Redmi Note 13	Mobile	Xiaomi	18999.00	50
7	Sony WH-1000XM5	Headphones	Sony	29999.00	10
4	OnePlus 12	Mobile	OnePlus	54999.00	25
8	Dell Inspiron 15	Laptop	Dell	58999.00	12
2	Galaxy S24	Mobile	Samsung	69999.00	15
1	iPhone 15	Mobile	Apple	79999.00	20

The bottom of the interface shows the 'Schema' section with 'music_streaming_app' selected, and an 'Output' section.

SESSION 6 - ORDER BY + Sorting

Task 2: Modify your previous query to show the top 5 most expensive products using ORDER BY with DESC and LIMIT.

Query / Answer:

```
select * from products order by price DESC limit 5;
```

Image / Screenshot:

The screenshot shows a SQL IDE interface. On the left is a sidebar with a tree view containing folders for 'MAS', 'classwork', 'music_streaming_app', 'product', and 'sys'. The 'product' folder is expanded, showing 'Tables', 'Views', 'Stored Procedures', and 'Functions'. The main editor area is titled 'Query 1' and contains the following SQL code:

```
18 ('Realme Narzo 70', 'Mobile', 'Realme', 15999.00, 40),
19 ('Boat Rockerz 450', 'Headphones', 'Boat', 1499.00, 100),
20 ('Sony WH-1000XMS', 'Headphones', 'Sony', 29999.00, 10),
21 ('Dell Inspiron 15', 'Laptop', 'Dell', 58999.00, 12);
22
23
24 • select * from products order by price DESC limit 5;
```

Below the query editor is the 'Result Grid' tab, which displays the results of the query in a table format. The table has columns: product_id, product_name, category, brand, price, and stock. The results are as follows:

product_id	product_name	category	brand	price	stock
1	iPhone 15	Mobile	Apple	79999.00	20
2	Galaxy S24	Mobile	Samsung	69999.00	15
8	Dell Inspiron 15	Laptop	Dell	58999.00	12
4	OnePlus 12	Mobile	OnePlus	54999.00	25
7	Sony WH-1000XMS	Headphones	Sony	29999.00	10
•	NULL	NULL	NULL	NULL	NULL

SESSION 6 - ORDER BY + Sorting

Task 3: Given a 'movies' table with columns 'title', 'release_year', and 'rating', write an SQL query to list all movies sorted first by release_year in descending order (latest first), then by rating in descending order (highest rated first).

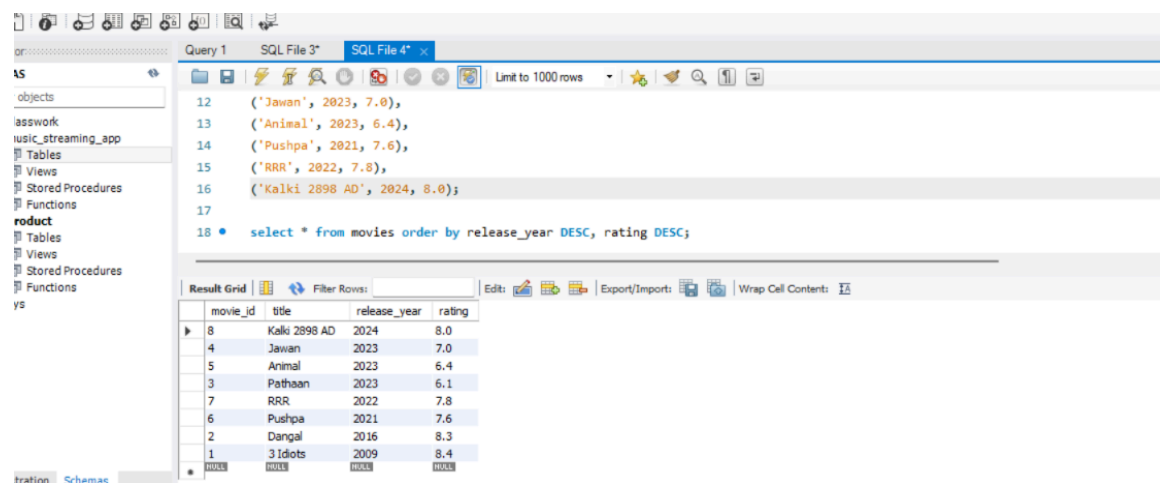
Query / Answer:

```
CREATE TABLE movies (  
  movie_id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(100),  
  release_year INT,  
  rating DECIMAL(3,1)  
);
```

```
INSERT INTO movies (title, release_year, rating) VALUES  
( '3 Idiots', 2009, 8.4),  
( 'Dangal', 2016, 8.3),  
( 'Pathaan', 2023, 6.1),  
( 'Jawan', 2023, 7.0),  
( 'Animal', 2023, 6.4),  
( 'Pushpa', 2021, 7.6),  
( 'RRR', 2022, 7.8),  
( 'Kalki 2898 AD', 2024, 8.0);
```

```
select * from movies order by release_year DESC, rating DESC;
```

Image / Screenshot:



The screenshot displays a SQL IDE interface. The query editor shows the following SQL code:

```
12 ('Jawan', 2023, 7.0),  
13 ('Animal', 2023, 6.4),  
14 ('Pushpa', 2021, 7.6),  
15 ('RRR', 2022, 7.8),  
16 ('Kalki 2898 AD', 2024, 8.0);  
17  
18 • select * from movies order by release_year DESC, rating DESC;
```

The result grid shows the following data:

movie_id	title	release_year	rating
8	Kalki 2898 AD	2024	8.0
4	Jawan	2023	7.0
5	Animal	2023	6.4
3	Pathaan	2023	6.1
7	RRR	2022	7.8
6	Pushpa	2021	7.6
2	Dangal	2016	8.3
1	3 Idiots	2009	8.4

SESSION 6 - ORDER BY + Sorting

--

SESSION 6 - ORDER BY + Sorting

Task 4: Write an SQL query to display the first 10 restaurants from a 'restaurants' table, sorted alphabetically by name, just like Zomato's A-Z listing. Hint: Use ORDER BY with LIMIT.

Query / Answer:

```
CREATE TABLE restaurants (  
  restaurant_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  cuisine VARCHAR(50),  
  location VARCHAR(100),  
  rating DECIMAL(2,1)  
);  
  
INSERT INTO restaurants (name, cuisine, location, rating) VALUES  
( 'Barbeque Nation', 'North Indian', 'Ahmedabad', 4.5),  
( 'Dominos Pizza', 'Italian', 'Vadodara', 4.2),  
( 'Havmor Restaurant', 'Gujarati', 'Ahmedabad', 4.3),  
( 'KFC', 'Fast Food', 'Surat', 4.1),  
( 'McDonalds', 'Fast Food', 'Vadodara', 4.0),  
( 'Pizza Hut', 'Italian', 'Rajkot', 4.2),  
( 'Subway', 'Healthy', 'Ahmedabad', 4.1),  
( 'The Grand Thakar', 'Gujarati', 'Ahmedabad', 4.6),  
( 'Udupi Restaurant', 'South Indian', 'Surat', 4.4),  
( 'Wow Momo', 'Chinese', 'Vadodara', 4.0),  
( 'Burger King', 'Fast Food', 'Ahmedabad', 4.1),  
( 'Sankalp', 'South Indian', 'Rajkot', 4.3);  
  
select * from restaurants order by name limit 10;
```

Image / Screenshot:

SESSION 6 - ORDER BY + Sorting

The screenshot shows a database IDE interface. On the left is a sidebar with a tree view of database objects. The main area is split into two panes. The top pane shows a SQL query editor with a query that inserts data into a 'restaurants' table and then selects all records ordered by name. The bottom pane shows the 'Result Grid' displaying the output of the query.

Query 1

```
18 ('Udupi Restaurant', 'South Indian', 'Surat', 4.4),
19 ('Wow Momo', 'Chinese', 'Vadodara', 4.0),
20 ('Burger King', 'Fast Food', 'Ahmedabad', 4.1),
21 ('Sankalp', 'South Indian', 'Rajkot', 4.3);
22
23
24 select * from restaurants order by name limit 10;
```

Result Grid

restaurant_id	name	cuisine	location	rating
1	Barbeque Nation	North Indian	Ahmedabad	4.5
11	Burger King	Fast Food	Ahmedabad	4.1
2	Dominos Pizza	Italian	Vadodara	4.2
3	Havmor Restaurant	Gujarati	Ahmedabad	4.3
4	KFC	Fast Food	Surat	4.1
5	McDonalds	Fast Food	Vadodara	4.0
6	Pizza Hut	Italian	Rajkot	4.2
12	Sankalp	South Indian	Rajkot	4.3
7	Subway	Healthy	Ahmedabad	4.1
8	The Grand Thakar	Gujarati	Ahmedabad	4.6
*	NULL	NULL	NULL	NULL

Schema:

restaurants 1 x

SESSION 6 - ORDER BY + Sorting

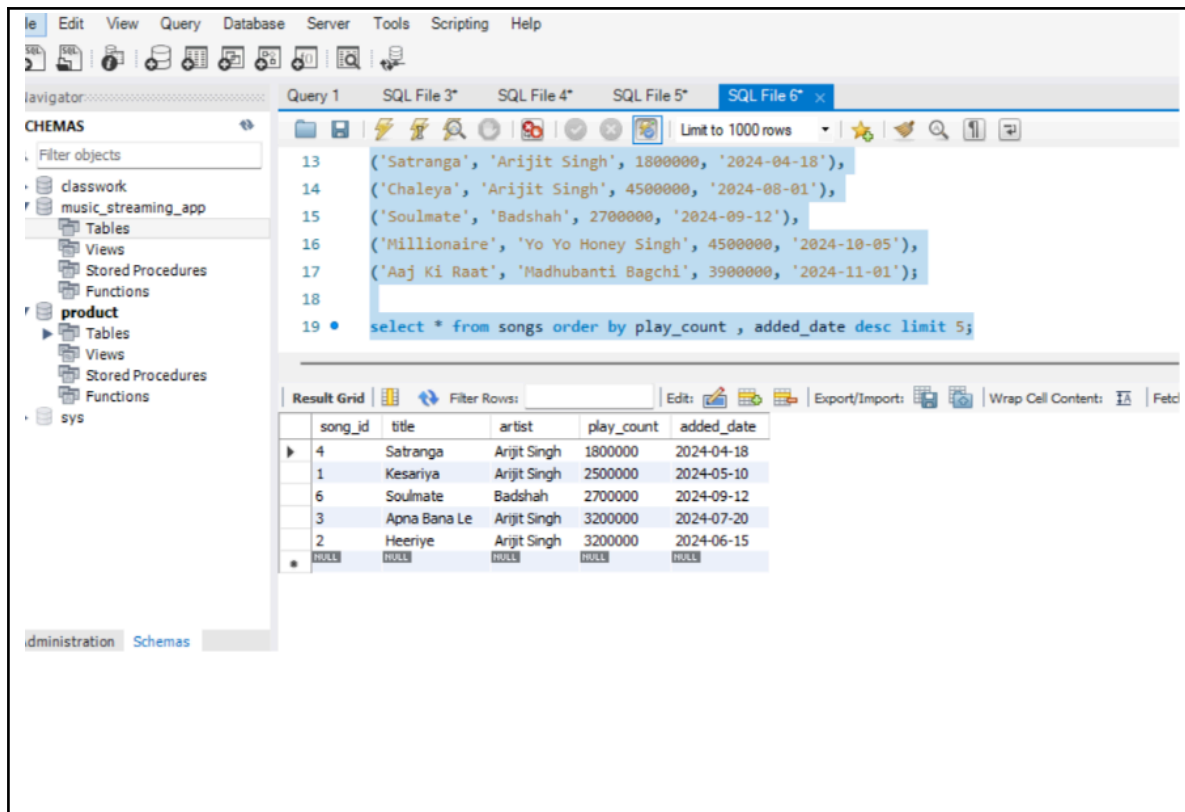
Task 5: Suppose you want to display the top 3 trending songs from a 'songs' table based on play_count, but if two songs have the same play_count, the more recently added song should come first. Write the SQL query to achieve this. Hint: Use ORDER BY with multiple columns.

Query / Answer:

```
CREATE TABLE songs (  
  song_id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(100),  
  artist VARCHAR(100),  
  play_count INT,  
  added_date DATE  
);  
  
INSERT INTO songs (title, artist, play_count, added_date) VALUES  
( 'Kesariya', 'Arijit Singh', 2500000, '2024-05-10'),  
( 'Heeriye', 'Arijit Singh', 3200000, '2024-06-15'),  
( 'Apna Bana Le', 'Arijit Singh', 3200000, '2024-07-20'),  
( 'Satranga', 'Arijit Singh', 1800000, '2024-04-18'),  
( 'Chaleya', 'Arijit Singh', 4500000, '2024-08-01'),  
( 'Soulmate', 'Badshah', 2700000, '2024-09-12'),  
( 'Millionaire', 'Yo Yo Honey Singh', 4500000, '2024-10-05'),  
( 'Aaj Ki Raat', 'Madhubanti Bagchi', 3900000, '2024-11-01');  
  
select * from songs order by play_count , added_date desc limit 5;
```

Image / Screenshot:

SESSION 6 - ORDER BY + Sorting



The screenshot displays a SQL IDE interface with a query editor and a result grid. The query editor shows a list of song data and a SQL query to sort by play_count and added_date. The result grid shows the top 5 songs sorted by play_count in descending order.

Query Editor:

```
13 ('Satranga', 'Arijit Singh', 1800000, '2024-04-18'),  
14 ('Chaleya', 'Arijit Singh', 4500000, '2024-08-01'),  
15 ('Soulmate', 'Badshah', 2700000, '2024-09-12'),  
16 ('Millionaire', 'Yo Yo Honey Singh', 4500000, '2024-10-05'),  
17 ('Aaj Ki Raat', 'Madhubanti Bagchi', 3900000, '2024-11-01');  
18  
19 • select * from songs order by play_count , added_date desc limit 5;
```

Result Grid:

song_id	title	artist	play_count	added_date
4	Satranga	Arijit Singh	1800000	2024-04-18
1	Kesariya	Arijit Singh	2500000	2024-05-10
6	Soulmate	Badshah	2700000	2024-09-12
3	Apna Bana Le	Arijit Singh	3200000	2024-07-20
2	Heeriye	Arijit Singh	3200000	2024-06-15
NULL	NULL	NULL	NULL	NULL